The granularity of a lock is something I touched on earlier, in section 3.2.3: the lock granularity is a hand-waving term to describe the amount of data protected by a single lock. A fine-grained lock protects a small amount of data, and a coarse-grained lock protects a large amount of data. Not only is it important to choose a sufficiently coarse lock granularity to ensure the required data is protected, but it's also important to ensure that a lock is held only for the operations that actually require it. We all know the frustration of waiting in the checkout line in a supermarket with a cart full of groceries only for the person currently being served to suddenly realize that they forgot some cranberry sauce and then leave everybody waiting while they go and find some, or for the cashier to be ready for payment and the customer to only then start rummaging in their purse for their wallet. Everything proceeds much more easily if everybody gets to the checkout with everything they want and with an appropriate means of payment ready.

The same applies to threads: if multiple threads are waiting for the same resource (the cashier at the checkout), then if any thread holds the lock for longer than necessary, it will increase the total time spent waiting (don't wait until you've reached the checkout to start looking for the cranberry sauce). Where possible, lock a mutex only while actually accessing the shared data; try to do any processing of the data outside the lock. In particular, don't do any really time-consuming activities like file I/O while holding a lock. File I/O is typically hundreds (if not thousands) of times slower than reading or writing the same volume of data from memory. So unless the lock is really intended to protect access to the file, performing I/O while holding the lock will delay *other* threads unnecessarily (because they'll block while waiting to acquire the lock), potentially eliminating any performance gain from the use of multiple threads.

`std::unique_lock` works well in this situation, because you can call `unlock()` when the code no longer needs access to the shared data and then call `lock()` again if access is required later in the code: